Linux File Permissions
Understanding rwx, chmod, and Numeric Modes

Lecture Notes and Practical Reference

Learning objectives
By the end of this handout, students should be able to interpret Linux permission strings, calculate numeric permission modes, inspect permissions, and modify permissions using the `chmod` command.

1. Introduction

Linux file permissions control who may read, modify, or execute files and who may access directories. Permissions are assigned to three classes: the file owner, the file's group, and other users on the system.

2. Basic permissions: r, w, and x

Symbol	Permission	Meaning	Numeric value
r	Read	View file contents; for directories, list entries (subject to directory permissions).	4
w	Write	Modify file contents; for directories, create, remove, or rename entries (subject to directory permissions).	2
x	Execute	Run a file; for directories, traverse/search the directory.	1
-	No permission	The corresponding permission is not granted.	0

A permission triplet is calculated by adding the numeric values of the permissions present. For example, `rwx` = 4 + 2 + 1 = 7; `r-x` = 4 + 0 + 1 = 5; and `r--` = 4 + 0 + 0 = 4.

3. The three permission classes

A nine-character permission string is divided into three groups of three characters:

Class	Position	Applies to
User (u)	First three characters	The owner of the file or directory.
Group (g)	Middle three characters	Users who belong to the file's assigned group.
Others (o)	Last three characters	All other users.

Example: `rwxr-xr--`

User	Group	Others
rwx = 7	r-x = 5	r-- = 4

Therefore, `rwxr-xr--` **= 754**. The first digit is the owner's permission, the second is the group's permission, and the third is the permissions for others.

4. Common numeric permission modes

Numeric mode	Symbolic form	Explanation
777	rwxrwxrwx	All users have read, write, and execute permissions.
755	rwxr-xr-x	Owner has full permissions; group and others can read and execute.
754	rwxr-xr--	Owner has full permissions; group can read and execute; others can read.
750	rwxr-x---	Owner has full permissions; group can read and execute; others have no permissions.
700	rwx------	Only the owner has full permissions.
666	rw-rw-rw-	All users can read and write; no execute permission.
644	rw-r--r--	Owner can read and write; group and others can read.
640	rw-r-----	Owner can read and write; group can read; others have no permissions.
600	rw-------	Only the owner can read and write.
444	r--r--r--	All users can read; no write or execute permission.

5. Calculating numeric permissions

Use the following values: read = 4, write = 2, execute = 1, and no permission = 0. Add the values in each three-character group separately to obtain the three-digit mode.

Symbolic triplet	Calculation	Numeric value
rwx	4 + 2 + 1	7
rw-	4 + 2 + 0	6
r-x	4 + 0 + 1	5
r--	4 + 0 + 0	4
-wx	0 + 2 + 1	3
-w-	0 + 2 + 0	2
--x	0 + 0 + 1	1
---	0 + 0 + 0	0

6. Inspecting file permissions

Use `ls -l` to display file permissions and related file information:

```
ls -l
-rwxr-xr-- 1 student students 1200 Nov 03 program.sh
```

The first character indicates the file type: `-` represents a regular file and `d` represents a directory. The following nine characters show user, group, and others permissions.

7. Changing permissions with chmod

The `chmod` command changes the permissions of a file or directory. Numeric mode and symbolic mode are two common methods.

A. Numeric method

```
chmod 755 program.sh
chmod 644 report.txt
chmod 700 private.sh
chmod 777 shared-file
```

B. Symbolic method

Command	Effect
chmod u+x file	Add execute permission for the owner.
chmod g+w file	Add write permission for the group.
chmod o-r file	Remove read permission from others.
chmod a+x file	Add execute permission for all classes.
chmod u-w file	Remove write permission from the owner.
chmod u=rwx file	Set the owner's permissions to exactly rwx.

Permission class letters: u = user/owner, g = group, o = others, and a = all. Operators: + adds, - removes, and = sets the specified permissions exactly.

8. Practical demonstration

The following sequence allows students to observe permission changes directly in the terminal:

```
# Create a sample file
touch practice.txt

# View its current permissions
ls -l practice.txt

# Set permissions to 755
chmod 755 practice.txt

# Verify the updated permissions
ls -l practice.txt

# Remove write permission from others
chmod o-w practice.txt

# Verify again
ls -l practice.txt
```

9. Student practice exercises

A. Convert each symbolic permission string into its numeric equivalent:

No.	Symbolic permission	Numeric value
1	rwxrwxrwx	_____
2	rw-r--r--	_____
3	rwxr-x---	_____
4	r--------	_____
5	rwx------	_____
6	rw-rw----	_____
7	r-xr-xr-x	_____
8	rw-------	_____
9	rwxr-----	_____
10	r--r-----	_____

B. Practical tasks:

1. Create a file named `student.txt` and assign permission mode 644.
2. Create a shell script and assign permission mode 755.
3. Remove write permission from the group of a selected file.
4. Give the owner execute permission without changing other permissions.
5. Set a file so that only its owner can read and write it.
6. Use `ls -l` to verify the permission changes after each task.

10. Good practices and security considerations

• Apply the minimum permissions required for a task.
• Avoid using `chmod 777` as a routine solution because it grants write access to everyone.
• Use quotes around file paths that may contain spaces.
• Verify permission changes with `ls -l`.
• Remember that directory permissions have distinct effects: read lists directory entries, write changes entries, and execute permits traversal/search.
• File ownership and permissions are separate concepts; changing permissions does not change the owner.

Conclusion

Linux permissions provide a fundamental access-control mechanism for files and directories. Students should be able to interpret the `rwx` notation, calculate numeric modes such as 755 and 644, and apply permission changes with `chmod`. Practical terminal exercises help reinforce these concepts.